



Tecnológico de Monterrey

Campus Monterrey

Evidencia1 pruebas

Diego Villanueva Fernandez
A01199495

David Alonso Cantú Delgado
Programación de estructuras de datos y algoritmos fundamentales
Grupo 607

20 de Sep del 2026

Plan de pruebas

El objetivo de las pruebas fue verificar que el programa pudiera leer ambos archivos de registros, ordenar correctamente 6,818 elementos con cada algoritmo disponible, medir el tiempo de ejecución, comparar el resultado con una predicción inicial y realizar búsquedas por rango de fecha/hora sin perder registros.

Se ejecutaron los siete algoritmos de ordenamiento sobre los dos archivos proporcionados: log607-1.txt (desordenado) y log607-2.txt (casi ordenado). Esto produjo 14 combinaciones de prueba. En cada corrida se registraron el algoritmo, archivo, número de registros, tiempo de ejecución, complejidad teórica y coincidencia de la predicción.

Casos funcionales adicionales

- Rango vacío: búsqueda fuera del intervalo real de los logs (1 de enero de 2030), esperando cero resultados.
- Límites inclusivos: el programa considera incluidos tanto el inicio como el final del rango.
- Timestamps duplicados: cuando el inicio o final coincide con un timestamp repetido, se incluyen todos los registros con ese timestamp.
- Formato inválido: se verificó que entradas de fecha/hora mal formadas fueran rechazadas y solicitadas nuevamente.
- Extremos cronológicos: después de ordenar, el primer registro fue Sep 08 2024 00:22:43 y el último Sep 07 2026 23:50:40.

Resultados de las 14 corridas

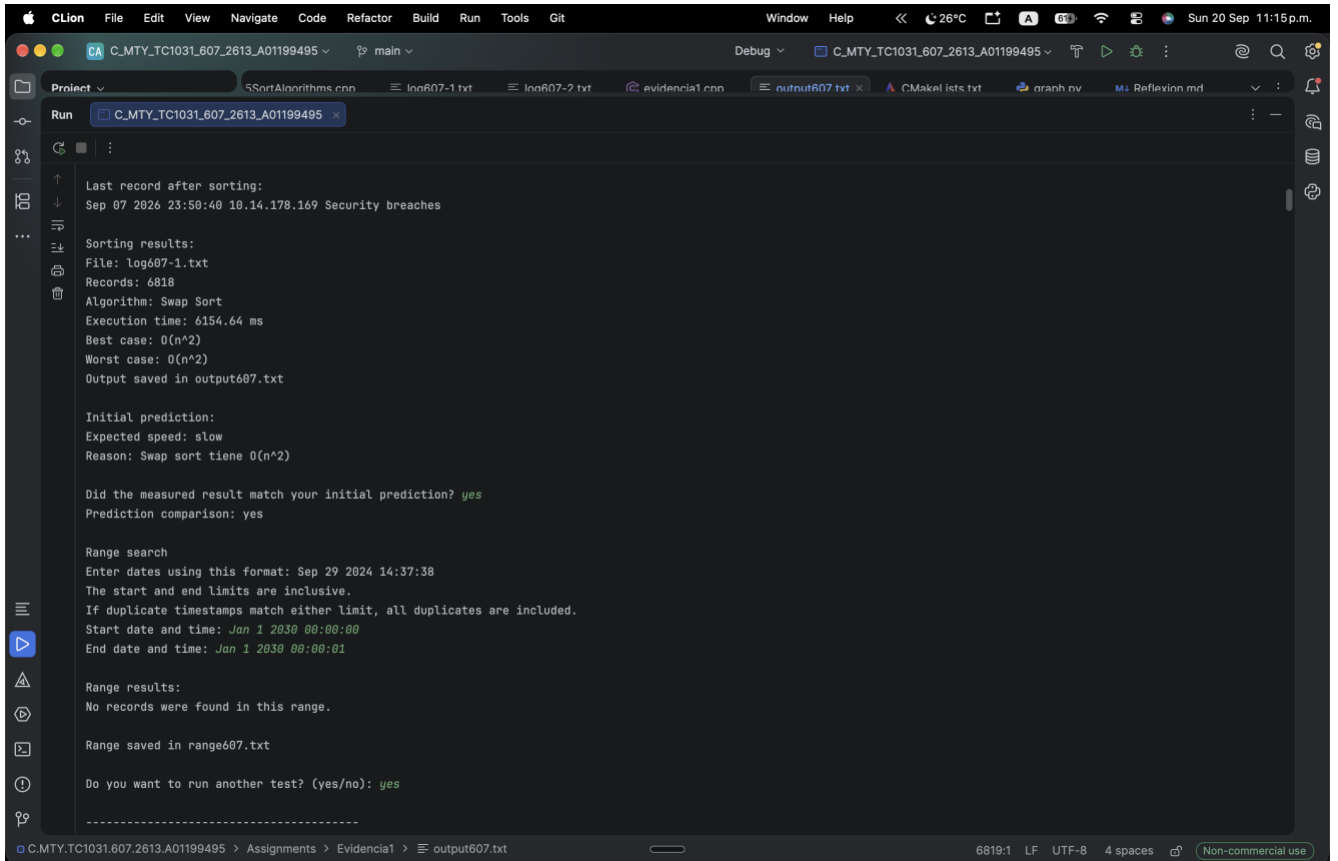
Algoritmo	log607-1 (ms)	log607-2 (ms)	Predicción 1 / 2
Swap Sort	6154.64	98.6084	slow / no
Bubble Sort	6062.38	108.137	slow / no
Selection Sort	107.4	106.951	slow / no
Insertion Sort	248.895	2.37004	medium / very fast
Merge Sort	40.219	39.0092	very fast / very fast
Quick Sort	51.8463	3053.51	fast / very slow
Shell Sort	17.7038	11.0785	fast / very fast

Los tiempos muestran que el estado inicial de los datos puede cambiar de manera importante el rendimiento práctico. Esto es especialmente visible en Insertion Sort y Quick Sort.

Evidencia de ejecución - log607-1.txt

El archivo log607-1.txt está desordenado. Las capturas siguientes muestran ejemplos representativos de algoritmos con comportamientos distintos sobre el mismo conjunto de 6,818 registros.

Swap Sort



```
CLion File Edit View Navigate Code Refactor Build Run Tools Git Window Help 26°C Sun 20 Sep 11:15 p.m.
C_MTY_TC1031_607_2613_A01199495 main
Project C_MTY_TC1031_607_2613_A01199495
5SortAlgorithms.cpp log607-1.txt log607-2.txt evidencia1.cpp output607.txt CMakelists.txt graph nv Reflexion.md
Run C_MTY_TC1031_607_2613_A01199495
Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-1.txt
Records: 6818
Algorithm: Swap Sort
Execution time: 6154.64 ms
Best case: O(n^2)
Worst case: O(n^2)
Output saved in output607.txt

Initial prediction:
Expected speed: slow
Reason: Swap sort tiene O(n^2)

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

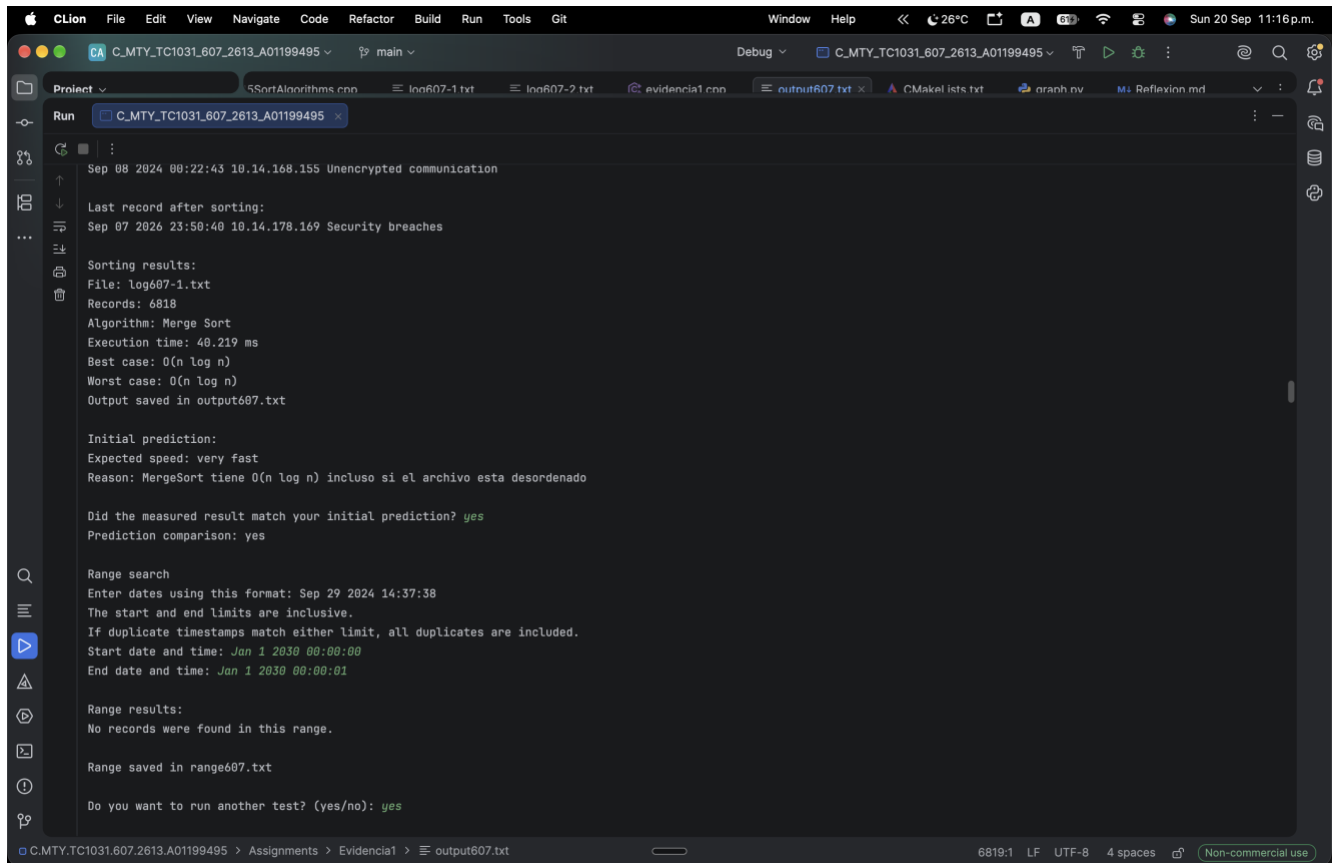
Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): yes
-----
C_MTY_TC1031_607_2613_A01199495 > Assignments > Evidencia1 > output607.txt 6819:1 LF UTF-8 4 spaces Non-commercial use
```

Figura 1. Swap Sort sobre log607-1.txt: 6154.64 ms, con complejidad $O(n^2)$.

Merge Sort



```
Project: 5SortAlgorithms.cpp | log607-1.txt | log607-2.txt | evidencia1.cpp | output607.txt | CMakelists.txt | graph.py | Reflexion.md
Run: C_MTY_TC1031_607_2613_A01199495

Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-1.txt
Records: 6818
Algorithm: Merge Sort
Execution time: 40.219 ms
Best case:  $O(n \log n)$ 
Worst case:  $O(n \log n)$ 
Output saved in output607.txt

Initial prediction:
Expected speed: very fast
Reason: MergeSort tiene  $O(n \log n)$  incluso si el archivo esta desordenado

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

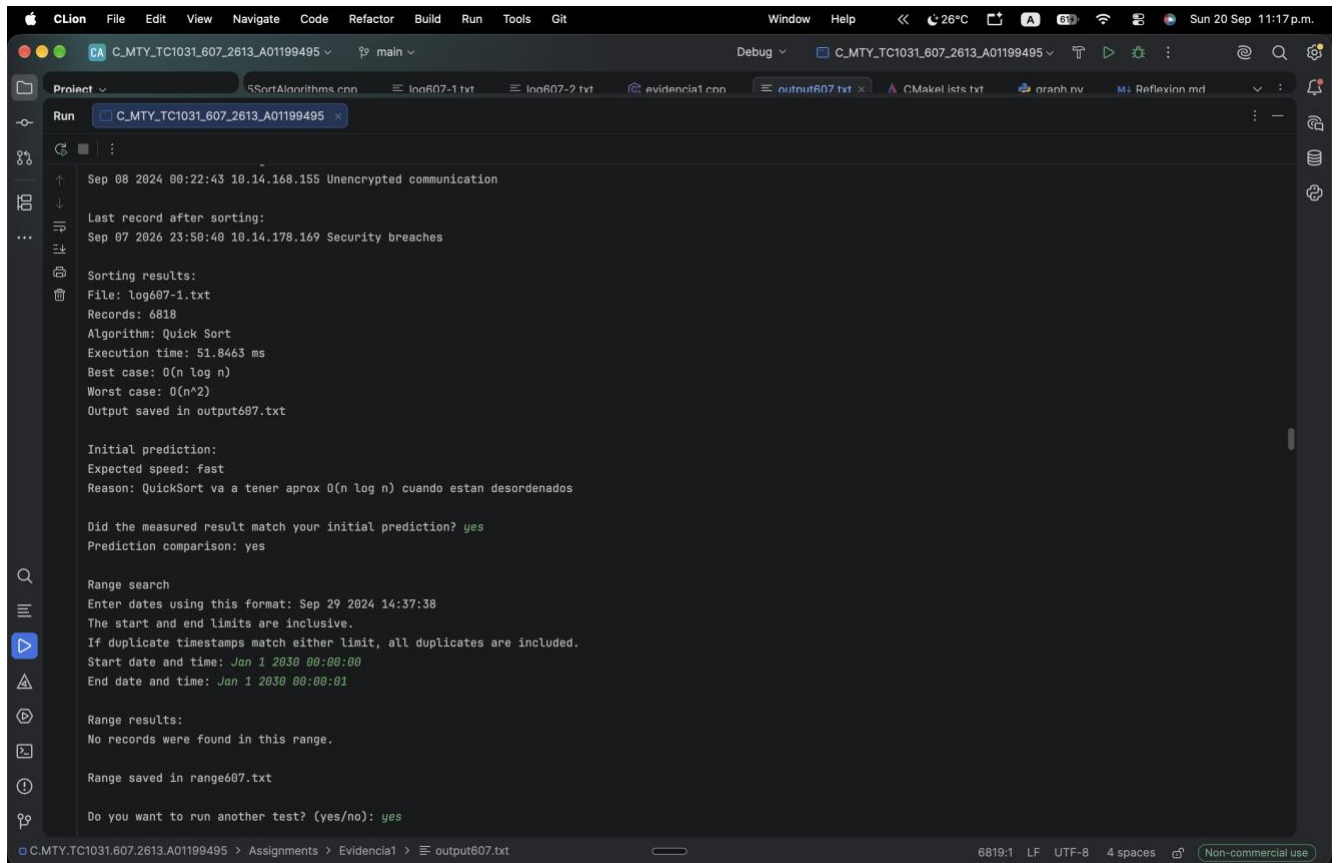
Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): yes
```

Figura 2. Merge Sort sobre log607-1.txt: 40.219 ms, con $O(n \log n)$ en mejor y peor caso.

Quick Sort



```
Project: SSortAlgorithms.cpp | log607-1.txt | log607-2.txt | evidencia1.cpp | output607.txt | CMakelists.txt | graph ov | M+ Reflexion.md
Run: C_MTY_TC1031_607_2613_A01199495

Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-1.txt
Records: 6818
Algorithm: Quick Sort
Execution time: 51.8463 ms
Best case:  $O(n \log n)$ 
Worst case:  $O(n^2)$ 
Output saved in output607.txt

Initial prediction:
Expected speed: fast
Reason: QuickSort va a tener aprox  $O(n \log n)$  cuando estan desordenados

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

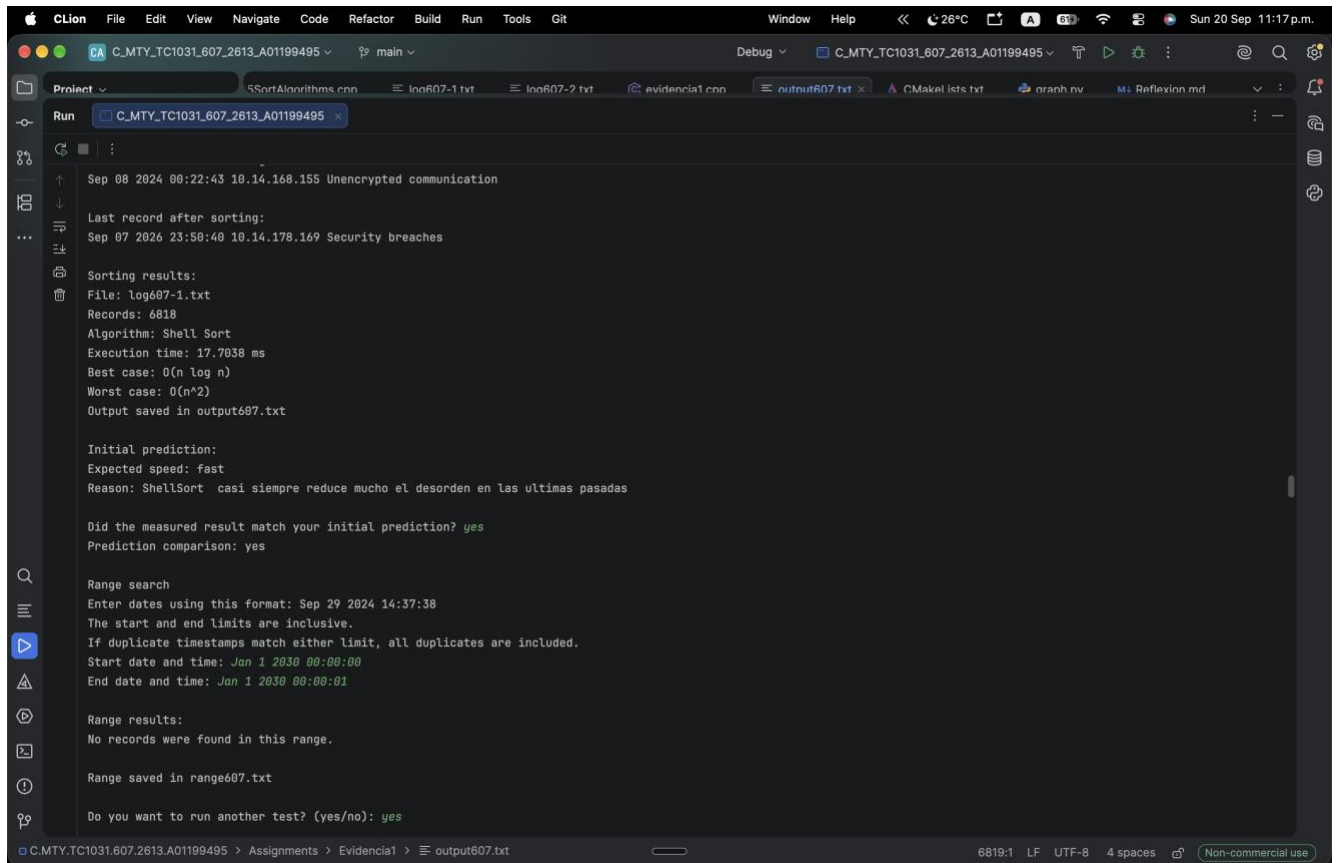
Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): yes
```

Figura 3. Quick Sort sobre log607-1.txt: 51.8463 ms. La predicción de ejecución rápida coincidió.

Shell Sort



```
CLion File Edit View Navigate Code Refactor Build Run Tools Git Window Help 26°C 6:17 Sun 20 Sep 11:17 p.m.
C_MTY_TC1031_607_2613_A01199495 main
Run C_MTY_TC1031_607_2613_A01199495
Project SSortAlgorithms.cpp log607-1.txt log607-2.txt evidenci1.cpp output607.txt CMakelists.txt graph ov M Reflexion.md

Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-1.txt
Records: 6818
Algorithm: Shell Sort
Execution time: 17.7038 ms
Best case: O(n log n)
Worst case: O(n^2)
Output saved in output607.txt

Initial prediction:
Expected speed: fast
Reason: ShellSort casi siempre reduce mucho el desorden en las ultimas pasadas

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): yes

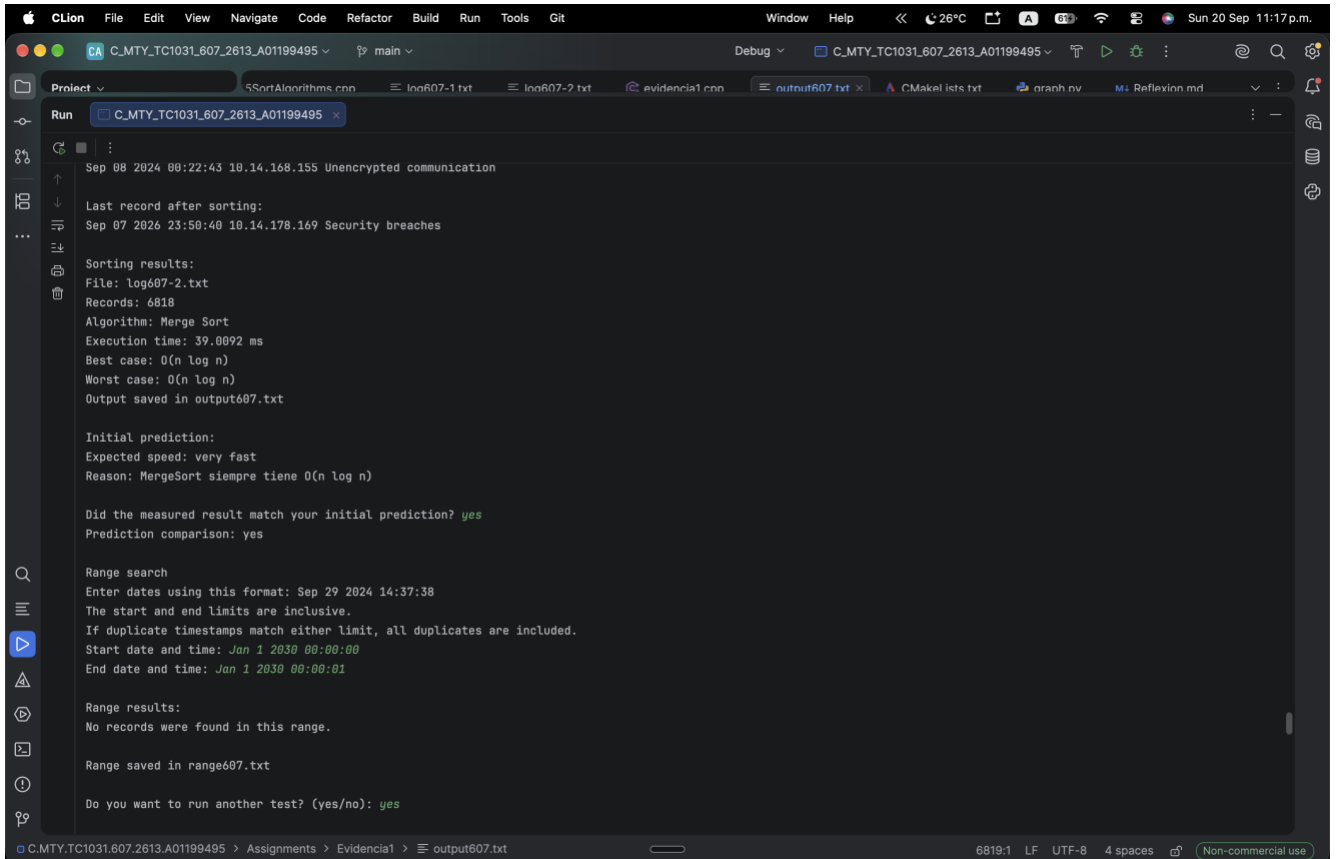
C_MTY_TC1031_607_2613_A01199495 > Assignments > Evidencia1 > output607.txt 6819:1 LF UTF-8 4 spaces Non-commercial use
```

Figura 4. Shell Sort sobre log607-1.txt: 17.7038 ms, el menor tiempo medido en el archivo desordenado.

Evidencia de ejecución - log607-2.txt

log607-2.txt está casi ordenado. Esto permitió observar de forma directa qué algoritmos aprovechan ese estado inicial y cuáles pueden empeorar por su estrategia de partición.

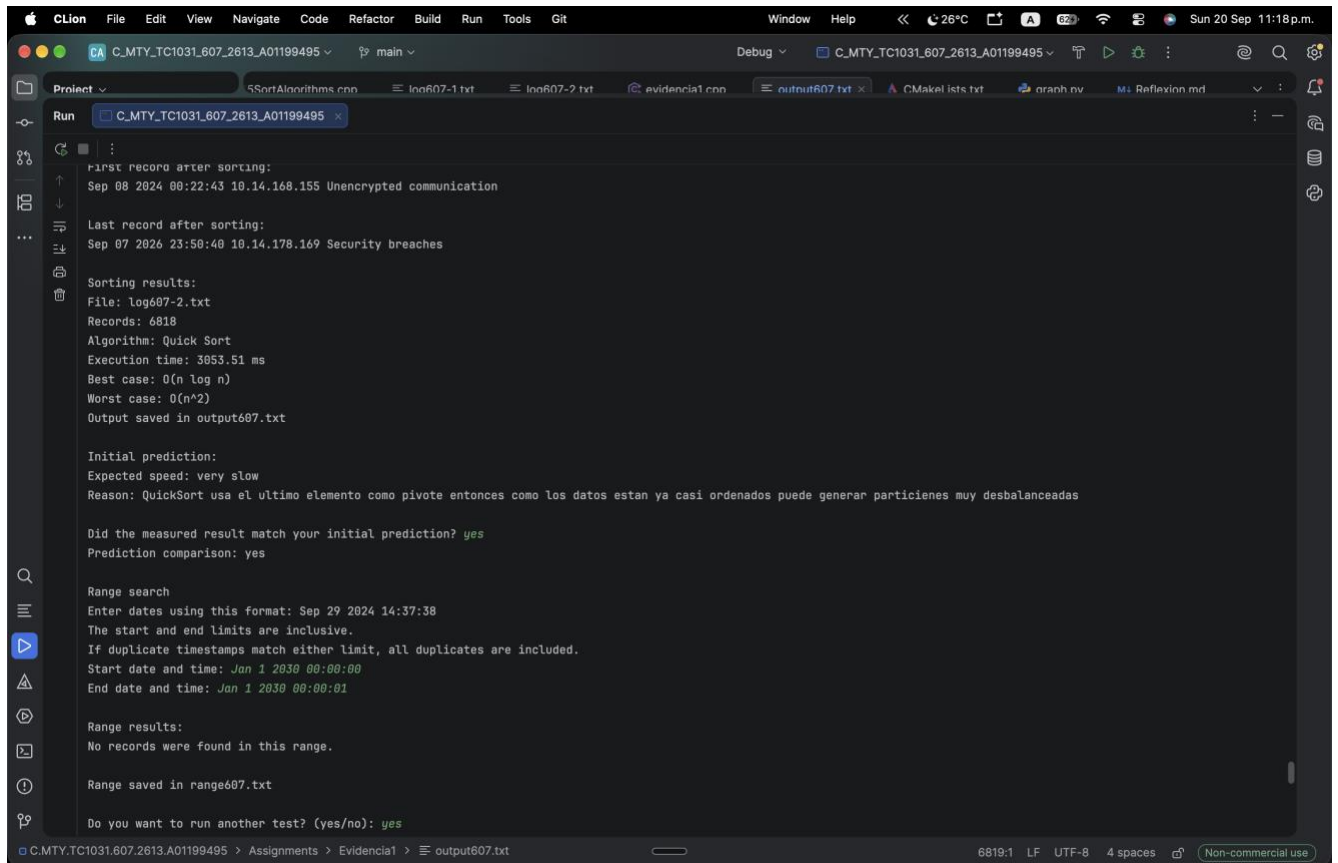
Insertion Sort



```
CLion File Edit View Navigate Code Refactor Build Run Tools Git Window Help 26°C Sun 20 Sep 11:17 p.m.
C_MTY_TC1031_607_2613_A01199495 main
Project C_MTY_TC1031_607_2613_A01199495
5SortAlgorithms.cpp log607-1.txt log607-2.txt evidencia1.cpp output607.txt CMakelists.txt graph nv Reflexion.md
Run C_MTY_TC1031_607_2613_A01199495
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches
Sorting results:
File: log607-2.txt
Records: 6818
Algorithm: Merge Sort
Execution time: 39.0092 ms
Best case: O(n log n)
Worst case: O(n log n)
Output saved in output607.txt
Initial prediction:
Expected speed: very fast
Reason: MergeSort siempre tiene O(n log n)
Did the measured result match your initial prediction? yes
Prediction comparison: yes
Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01
Range results:
No records were found in this range.
Range saved in range607.txt
Do you want to run another test? (yes/no): yes
C_MTY_TC1031_607_2613_A01199495 > Assignments > Evidencia1 > output607.txt 6819:1 LF UTF-8 4 spaces Non-commercial use
```

Figura 5. Insertion Sort sobre log607-2.txt: 2.37004 ms. Fue la corrida más rápida de las 14 pruebas.

Quick Sort



```
Project: SSortAlgorithms.cpp | log607-1.txt | log607-2.txt | evidencia1.cpp | output607.txt | CMakelists.txt | graph ov | M+ Reflexion.md
Run: C_MTY_TC1031_607_2613_A01199495

first record after sorting:
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Last record after sorting:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-2.txt
Records: 6818
Algorithm: Quick Sort
Execution time: 3053.51 ms
Best case:  $O(n \log n)$ 
Worst case:  $O(n^2)$ 
Output saved in output607.txt

Initial prediction:
Expected speed: very slow
Reason: QuickSort usa el ultimo elemento como pivote entonces como los datos estan ya casi ordenados puede generar particiones muy desbalanceadas

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

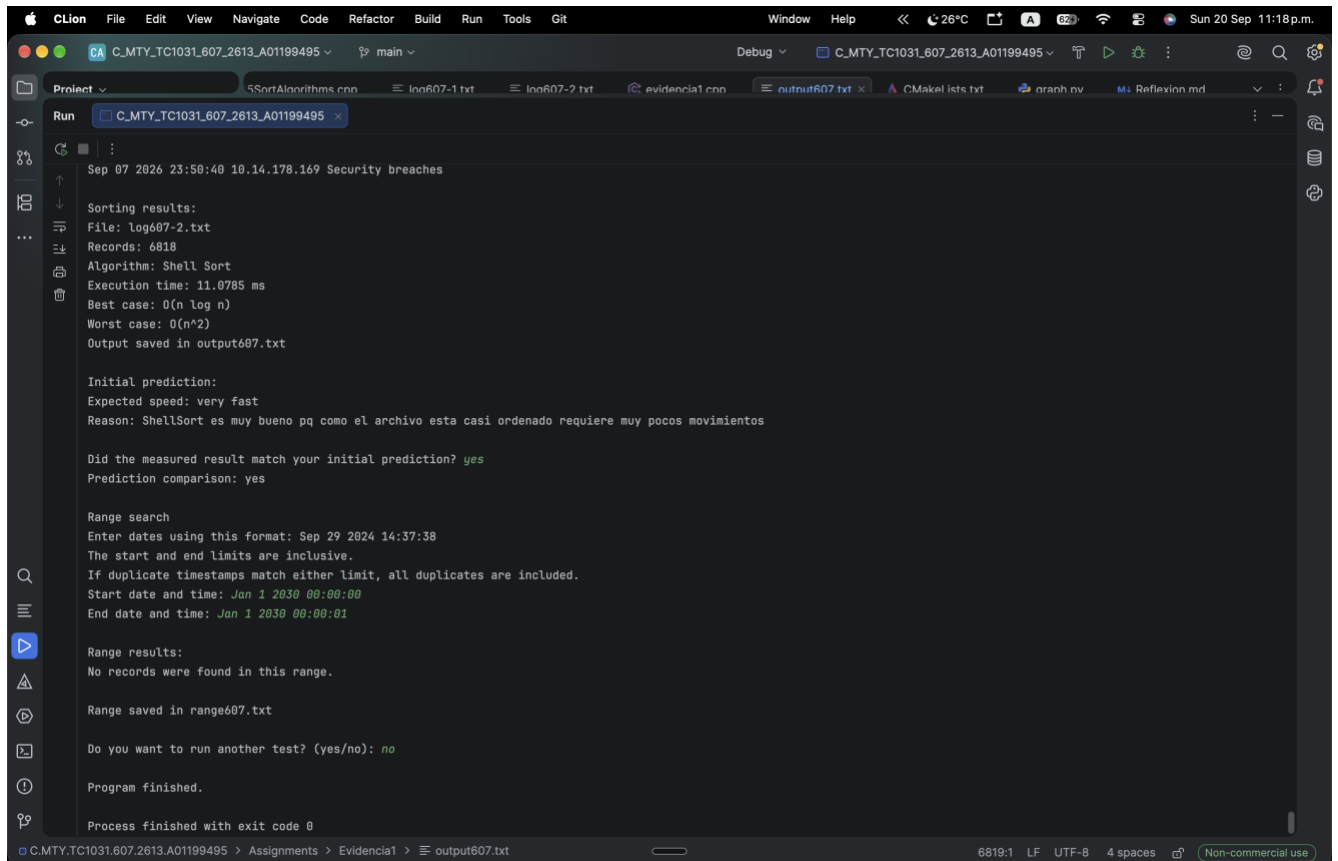
Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): yes
```

Figura 6. Quick Sort sobre log607-2.txt: 3053.51 ms. El pivote final provocó particiones muy desbalanceadas en datos casi ordenados.

Shell Sort



```
Project: SSortAlgorithms.cpp | log607-1.txt | log607-2.txt | evidencia1.cpp | output607.txt | CMakelists.txt | graph.py | Reflexion.md
Run: C_MTY_TC1031_607_2613_A01199495

Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Sorting results:
File: log607-2.txt
Records: 6818
Algorithm: Shell Sort
Execution time: 11.0785 ms
Best case: O(n log n)
Worst case: O(n^2)
Output saved in output607.txt

Initial prediction:
Expected speed: very fast
Reason: ShellSort es muy bueno pq como el archivo esta casi ordenado requiere muy pocos movimientos

Did the measured result match your initial prediction? yes
Prediction comparison: yes

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Jan 1 2030 00:00:00
End date and time: Jan 1 2030 00:00:01

Range results:
No records were found in this range.

Range saved in range607.txt

Do you want to run another test? (yes/no): no

Program finished.
Process finished with exit code 0
```

Figura 7. Shell Sort sobre log607-2.txt: 11.0785 ms. La ejecución finalizó correctamente con exit code 0.

Pruebas de búsqueda por rango

Rango vacío

Durante las corridas se utilizó el intervalo Jan 1 2030 00:00:00 a Jan 1 2030 00:00:01. Como los registros abarcan de 2024 a 2026, el programa devolvió “No records were found in this range”, validando el caso de rango vacío. La misma captura anterior también muestra que el archivo range607.txt fue generado.

Timestamps duplicados

Se probó el timestamp exacto Oct 02 2024 23:04:24 como inicio y fin. El programa devolvió dos registros distintos con la misma fecha y hora, confirmando que lower bound y upper bound incluyen todos los duplicados que coinciden con los límites:

Oct	02	2024	23:04:24	10.14.47.219	Social	engineering	attacks
Oct	02	2024	23:04:24	10.14.121.223	Changes	in	File Integrity

La política del programa es inclusiva: si un timestamp duplicado coincide con cualquiera de los límites, se conservan todas sus apariciones, sin pérdidas ni duplicaciones adicionales.

Pruebas adicionales de búsqueda

Formato inválido

```
Did the measured result match your initial prediction? 28 sept 11:37
Prediction comparison: 28 sept 11:37

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: 28 sept 11:37
Invalid date format. Try again.
Start date and time: |
```

Figura 8. El programa rechaza una entrada de fecha/hora con formato incorrecto y solicita nuevamente el dato.

Extremo cronológico inferior

```
Invalid date format. Try again.
Start date and time: Sep 08 2024 00:22:43
End date and time: Sep 08 2024 00:22:43

Range results:
Records found: 1

Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Range saved in range607.txt

Do you want to run another test? (yes/no):
```

Figura 9. La búsqueda inclusiva del primer timestamp devuelve exactamente el primer registro cronológico y lo guarda en range607.txt.

Pruebas adicionales de búsqueda

Extremo cronológico superior

```
Did the measured result match your initial prediction? Sep 07 2026 23:50:40
Prediction comparison: Sep 07 2026 23:50:40

Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Sep 07 2026 23:50:40
End date and time: Sep 07 2026 23:50:40

Range results:
Records found: 1

Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Range saved in range607.txt

Do you want to run another test? (yes/no):
```

Figura 10. La búsqueda inclusiva del último timestamp devuelve exactamente el último registro cronológico.

Timestamps duplicados

```
Range search
Enter dates using this format: Sep 29 2024 14:37:38
The start and end limits are inclusive.
If duplicate timestamps match either limit, all duplicates are included.
Start date and time: Oct 02 2024 23:04:24
End date and time: Oct 02 2024 23:04:24

Range results:
Records found: 2

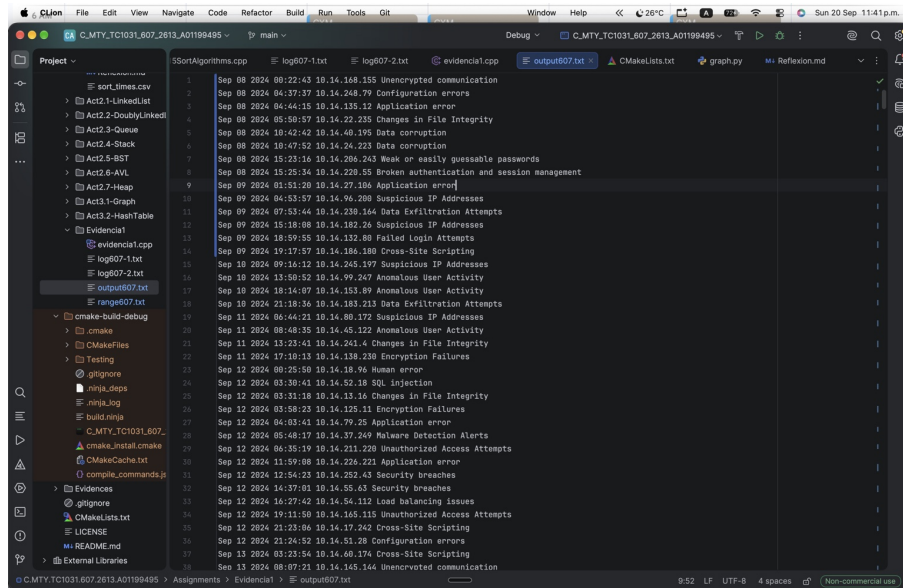
Oct 02 2024 23:04:24 10.14.47.219 Social engineering attacks
Oct 02 2024 23:04:24 10.14.121.223 Changes in File Integrity

Range saved in range607.txt

Do you want to run another test? (yes/no): |
```

Figura 11. Al buscar el mismo timestamp como inicio y fin, se recuperan los dos registros duplicados sin pérdidas.

output607.txt

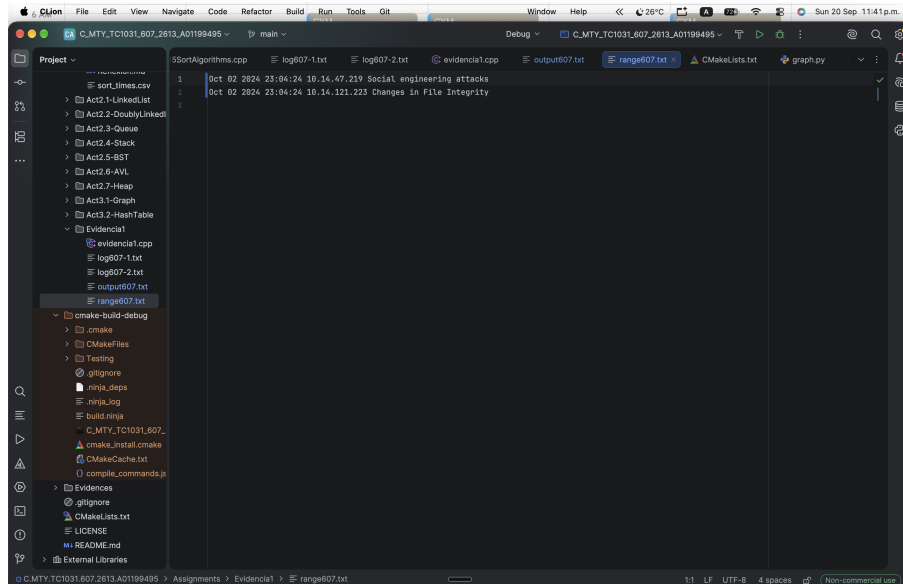


The screenshot shows a code editor with a project structure on the left. The main editor displays a log file with entries sorted chronologically. The entries are as follows:

Timestamp	IP Address	Event
Sep 08 2024 08:22:43	10.14.168.155	Unencrypted communication
Sep 08 2024 04:37:37	10.14.248.79	Configuration errors
Sep 08 2024 04:44:15	10.14.135.12	Application error
Sep 08 2024 05:50:57	10.14.22.235	Changes in File Integrity
Sep 08 2024 18:42:42	10.14.48.195	Data corruption
Sep 08 2024 10:47:52	10.14.44.223	Data corruption
Sep 08 2024 15:23:16	10.14.286.243	Weak or easily guessable passwords
Sep 08 2024 15:25:34	10.14.228.55	Broken authentication and session management
Sep 09 2024 01:51:20	10.14.27.184	Application error
Sep 09 2024 04:53:57	10.14.96.288	Suspicious IP Addresses
Sep 09 2024 07:53:44	10.14.238.164	Data Exfiltration Attempts
Sep 09 2024 15:18:08	10.14.182.28	Suspicious IP Addresses
Sep 09 2024 18:59:55	10.14.132.88	Failed Login Attempts
Sep 09 2024 19:17:57	10.14.186.188	Cross-Site Scripting
Sep 10 2024 09:16:12	10.14.245.197	Suspicious IP Addresses
Sep 10 2024 13:50:52	10.14.99.247	Anomalous User Activity
Sep 10 2024 18:14:07	10.14.183.89	Anomalous User Activity
Sep 10 2024 21:18:36	10.14.183.213	Data Exfiltration Attempts
Sep 11 2024 06:44:21	10.14.80.172	Suspicious IP Addresses
Sep 11 2024 08:48:35	10.14.45.122	Anomalous User Activity
Sep 11 2024 13:23:41	10.14.241.4	Changes in File Integrity
Sep 11 2024 17:18:13	10.14.138.238	Encryption Failures
Sep 12 2024 08:25:58	10.14.18.96	Human error
Sep 12 2024 03:38:41	10.14.52.18	SQL injection
Sep 12 2024 03:31:18	10.14.13.16	Changes in File Integrity
Sep 12 2024 03:58:23	10.14.125.11	Encryption Failures
Sep 12 2024 04:83:41	10.14.79.25	Application error
Sep 12 2024 05:48:17	10.14.37.949	Malware Detection Alerts
Sep 12 2024 06:35:19	10.14.211.228	Unauthorized Access Attempts
Sep 12 2024 11:59:88	10.14.226.221	Application error
Sep 12 2024 12:54:23	10.14.252.43	Security breaches
Sep 12 2024 14:37:01	10.14.55.63	Security breaches
Sep 12 2024 16:27:42	10.14.54.112	Load balancing issues
Sep 12 2024 19:11:58	10.14.165.415	Unauthorized Access Attempts
Sep 12 2024 21:23:86	10.14.17.242	Cross-Site Scripting
Sep 12 2024 21:24:52	10.14.51.28	Configuration errors
Sep 13 2024 03:23:54	10.14.60.174	Cross-Site Scripting
Sep 13 2024 08:07:21	10.14.145.144	Unencrypted communication

Figura 12. Fragmento de output607.txt con los registros ordenados cronológicamente y conservando el formato original.

range607.txt



The screenshot shows a code editor with a project structure on the left. The main editor displays a log file with two entries selected. The entries are as follows:

Timestamp	IP Address	Event
Oct 02 2024 23:04:24	10.14.47.219	Social engineering attacks
Oct 02 2024 23:04:24	10.14.121.223	Changes in File Integrity

Figura 13. Fragmento de range607.txt con los dos registros correspondientes al timestamp duplicado seleccionado.

Análisis de resultados

En log607-1.txt, Swap Sort y Bubble Sort fueron los más lentos, con 6154.64 ms y 6062.38 ms. Ambos realizan trabajo cuadrático y, además, ejecutan una gran cantidad de intercambios cuando el archivo está desordenado. Selection Sort también es $O(n^2)$, pero midió 107.4 ms; esto demuestra que Big-O describe el crecimiento asintótico y no determina por sí solo el tiempo real, ya que constantes, número de movimientos y costo de copiar registros también influyen.

Insertion Sort cambió de 248.895 ms en el archivo desordenado a 2.37004 ms en el archivo casi ordenado. Este resultado coincide con su mejor caso $O(n)$: cuando los elementos están cerca de su posición correcta, se requieren muy pocos desplazamientos.

Merge Sort fue estable entre ambos archivos: 40.219 ms y 39.0092 ms. Esto coincide con su comportamiento $O(n \log n)$, prácticamente independiente del orden inicial de los datos.

Quick Sort presentó el contraste más fuerte: 51.8463 ms en log607-1.txt y 3053.51 ms en log607-2.txt. La implementación utiliza el último elemento como pivote; con datos casi ordenados, esto puede generar particiones extremadamente desbalanceadas y acercar la ejecución al peor caso $O(n^2)$.

Shell Sort obtuvo 17.7038 ms y 11.0785 ms, mostrando buen desempeño en ambos escenarios y una mejora adicional cuando los registros ya estaban casi ordenados.

Conclusiones

Las pruebas confirmaron que los siete algoritmos ordenan correctamente los 6,818 registros y que el programa mantiene el formato original de salida. También se comprobó que las búsquedas por rango funcionan con límites inclusivos y que el manejo de timestamps duplicados conserva todos los registros correspondientes.

Los resultados experimentales muestran que la complejidad teórica debe interpretarse junto con la estructura de los datos. Insertion Sort fue especialmente adecuado para el archivo casi ordenado, mientras que la estrategia de pivote de Quick Sort resultó desfavorable en ese mismo escenario. Merge Sort mantuvo un rendimiento consistente y Shell Sort mostró los menores tiempos entre los algoritmos generales probados.

Por lo tanto, para registros de seguridad, la elección del algoritmo no debe basarse únicamente en su notación Big-O: también debe considerar qué tan ordenados están los datos, la estrategia de implementación y el costo real de comparaciones y movimientos.